

MODUL PRAKTIKUM 04

TOPIK: SIMULASI ALGORITMA PENJADWALAN CPU & GANTT CHART

Mata Kuliah: Sistem Operasi (INF-201) – Alokasi Waktu: 170 Menit

Program Studi Informatika – Fakultas Teknik – Universitas Bengkulu

1. CAPAIAN PEMBELAJARAN MODUL (SUB-CPMK 4)

Setelah menyelesaikan Modul Praktikum 04 ini, mahasiswa diharapkan mampu:

1. Mengkalkulasi metrik evaluasi penjadwalan CPU: *Turnaround Time* (TAT) dan *Waiting Time* (W).
2. Mensimulasikan eksekusi algoritma penjadwalan FCFS, SJF, dan Round Robin dalam kode program.
3. Membandingkan dampak ukuran *Time Quantum* q terhadap performa Round Robin dan MLFQ.

2. PRASYARAT TUGAS MEMBACA

Sebelum memulai percobaan laboratorium ini, mahasiswa **wajib** telah membaca bab referensi berikut:

- [OSTEP] Chapter 7: *Scheduling: Introduction* (hal. 61–72).
- [OSTEP] Chapter 8: *Scheduling: The Multi-Level Feedback Queue* (hal. 73–84).

3. LANDASAN TEORI RINGKAS

3.1 Metrik Evaluasi Penjadwalan CPU

Dua metrik kuantitatif utama untuk mengevaluasi performa algoritma penjadwalan CPU:

1. **Turnaround Time** (TAT): Total durasi dari waktu submit/kedatangan hingga proses selesai:

$$TAT_i = T_{\text{selesai},i} - T_{\text{datang},i}$$

2. **Waiting Time** (W): Total durasi proses berada di *Ready Queue* menunggu dialokasikan CPU:

$$W_i = TAT_i - \text{Burst Time}_i$$

3.2 5 Aturan Utama Multi-Level Feedback Queue (MLFQ)

1. **Rule 1:** Jika $\text{Priority}(A) > \text{Priority}(B)$, A berjalan (B tidak).
2. **Rule 2:** Jika $\text{Priority}(A) = \text{Priority}(B)$, A & B berjalan dalam Round Robin.
3. **Rule 3:** Saat proses baru memasuki sistem, diletakkan di antrean prioritas tertinggi.
4. **Rule 4:** Setelah proses menghabiskan jatah waktunya di satu tingkat, prioritasnya diturunkan 1 tingkat.
5. **Rule 5 (Priority Boost):** Setelah periode S , pindahkan **semua** proses ke antrean teratas.

4. KODE ACUAN UTAH (EXECUTABLE C LISTING: fcfs_sim.c)

```
1  #include <stdio.h>
2
3  struct Process {
4      int pid;
5      int arrival_time;
6      int burst_time;
7      int waiting_time;
8      int turnaround_time;
9  };
10
11 void calculate_times(struct Process proc[], int n) {
12     int current_time = 0;
13     for (int i = 0; i < n; i++) {
14         // Jika CPU menganggur menunggu kedatangan proses berikutnya
15         if (current_time < proc[i].arrival_time) {
16             current_time = proc[i].arrival_time;
17         }
18         proc[i].waiting_time = current_time - proc[i].arrival_time;
19         current_time += proc[i].burst_time;
20         proc[i].turnaround_time = proc[i].waiting_time + proc[i].burst_time;
21     }
22 }
23
24 int main() {
25     // Studi kasus 4 proses dengan waktu kedatangan non-zero
26     struct Process proc[] = {
27         {1, 0, 8, 0, 0},
28         {2, 1, 4, 0, 0},
29         {3, 2, 9, 0, 0},
30         {4, 3, 5, 0, 0}
31     };
32     int n = sizeof(proc) / sizeof(proc[0]);
33
34     calculate_times(proc, n);
35
36     printf("PID\tArrival\tBurst\tWaiting\tTurnaround\n");
37     float total_w = 0, total_tat = 0;
38     for (int i = 0; i < n; i++) {
39         total_w += proc[i].waiting_time;
40         total_tat += proc[i].turnaround_time;
41         printf("P%d\t%d\t%d\t%d\t%d\n", proc[i].pid, proc[i].arrival_time,
42             proc[i].burst_time, proc[i].waiting_time, proc[i].
43             turnaround_time);
44     }
45
46     printf("\nRata-rata Waiting Time: %.2f ms\n", total_w / n);
47     printf("Rata-rata Turnaround Time: %.2f ms\n", total_tat / n);
48     return 0;
49 }
```

Listing 1: Program Simulasi Penjadwalan FCFS C (fcfs_sim.c)

5. PROSEDUR LANGKAH KERJA PERCOBAAN

1. Buat berkas fcfs_sim.c dan salin kode acuan di atas ke dalam teks editor (VS Code / nano).

2. Kompilasi program menggunakan `gcc -Wall -O2 fcfs_sim.c -o fcfs_sim`.
3. Jalankan program dengan `./fcfs_sim` dan amati tabel keluaran (PID, Arrival, Burst, Waiting, Turnaround).
4. Uji kondisi CPU *idle*: Ubah waktu kedatangan P_2 menjadi $T_{\text{datang}} = 12$ (tiba setelah P_1 selesai pada $t = 8$). Amati bagaimana logika `if (current_time < proc[i].arrival_time)` menangani jeda waktu menganggur CPU secara tepat!

6. PANDUAN TROUBLESHOOTING ERROR UMUM

Pesan Error / Kendala	Penyebab Utama	Solusi Solutif
Rata-rata waktu bertipe integer (hasil bulat).	Pembagian variabel <code>total_w / n</code> dilakukan dalam integer.	Casting variabel menjadi float: <code>(float)total_w / n</code> .
Waktu tunggu bernilai negatif.	Proses belum tiba saat CPU dialokasikan ($T_{\text{datang}} > \text{current_time}$).	Pastikan menerapkan blok <code>current_time = proc[i].arrival_time</code> jika CPU idle.

7. TUGAS PRAKTIKUM & PERTANYAAN ANALISIS LAPORAN

1. **Pengembangan Kode SJF Non-Preemptive (`sjf_sim.c`):** Modifikasi kode C di atas sehingga mampu menyortir proses yang berada di *Ready Queue* berdasarkan nilai `burst_time` terpendek!
2. **Implementasi Round Robin Sederhana (`rr_sim.c`):** Rancang program simulasi Round Robin dengan kuantum $q = 3$ menggunakan atribut `remaining_burst` untuk melacak sisa eksekusi proses!
3. **Analisis Evaluasi MLFQ:** Mengapa Aturan ke-4 (*Accounting*) dan Aturan ke-5 (*Priority Boost*) pada MLFQ sangat vital pada sistem operasi komputer nyata?